

Análisis de Algoritmos de Ordenamiento sobre Arreglos Paginados con Política FIFO

Isaac Arias Picado

Estudiante de Ingeniería en Computadores

Tecnológico de Costa Rica

Cartago, Costa Rica

Email: i.arias.2@estudiantec.cr

Repositorio: <https://github.com/Isaac-Arias-Picado/Proyecto-Datos-2-Generator-Sorted.git>

Abstract—Este paper con fines académicos presenta el diseño e implementación de un sistema de ordenamiento basado en arreglos paginados para gestionar grandes volúmenes de datos que exceden la capacidad de memoria RAM. Se implementan cinco algoritmos (QuickSort, MergeSort, TimSort, RadixSort y ShellSort) sobre una estructura que simula memoria virtual con política de reemplazo FIFO. Se analiza el impacto de tres tamaños de página (16K, 64K y 256K enteros, equivalentes a 64 KB, 256 KB y 1 MB) en page faults, page hits, hit ratio y tiempo de ejecución para datasets de 256 MB (~67 millones de enteros), 512 MB (~134 millones) y 1 GB (~268 millones). Adicionalmente, se establece un baseline ejecutando cada algoritmo directamente en RAM para cuantificar el overhead de la paginación. Las pruebas se realizaron en un equipo Lenovo Legion 5i con procesador Intel Core Ultra 7, 16 GB de RAM DDR5 a 5600 MHz y SSD NVMe PCIe 4.0. Los resultados muestran que RadixSort es el más rápido en tiempo absoluto en todos los escenarios, aunque su overhead sobre el baseline alcanza $3.29\times$ en el dataset de 1 GB. ShellSort genera el mayor número de faults (~47× más que RadixSort en la peor configuración), mientras que aumentar el page size reduce faults drásticamente (hasta $\sim 40\times$).

Index Terms—Paged array, virtual memory, sorting algorithms, page faults, FIFO replacement, baseline comparison

I. INTRODUCCIÓN

El procesamiento de grandes volúmenes de datos es un desafío fundamental en la informática moderna. Cuando un dataset excede la capacidad de la memoria principal, el sistema operativo recurre a la memoria virtual paginada: los datos residen en disco y se cargan en RAM por bloques de tamaño fijo. Cada acceso a una página que no se encuentra en memoria genera un page fault, con latencias que pueden alcanzar millones de ciclos de reloj, en contraste con los 100–300 ciclos típicos de acceso a RAM.

Distintos algoritmos exhiben patrones de acceso radicalmente diferentes. Este trabajo evalúa empíricamente cinco algoritmos bajo distintas configuraciones de paginación sobre archivos de hasta 1 GB. Objetivos:

- 1) Implementar PagedArray: arreglo virtual sobre archivo binario en disco con caché en RAM.
- 2) Implementar cinco algoritmos de ordenamiento sobre esta estructura.
- 3) Establecer un baseline ejecutando los mismos algoritmos directamente en RAM.

- 4) Analizar el impacto del tamaño de página en page faults, hits, hit ratio y tiempo de ejecución para datasets de 256 MB, 512 MB y 1 GB.
- 5) Aproximar empíricamente la curva de crecimiento de cada algoritmo (tiempo vs. N) bajo condiciones de paginación.

II. BASELINE Y COMPARATIVA

A. Implementación del Baseline

Para establecer una referencia de rendimiento y cuantificar el overhead del sistema PagedArray, se ejecutó cada uno de los cinco algoritmos directamente sobre los datos cargados en RAM, sin restricciones artificiales de paginación. Este baseline representa las condiciones óptimas del hardware específico y permite medir el costo real que introduce la capa de simulación de memoria virtual.

B. Configuración del Baseline

Se ejecutó cada algoritmo sobre archivos de tres tamaños (Tabla I), midiendo el tiempo de ejecución en condiciones de memoria sin restricción.

TABLE I
CONJUNTOS DE DATOS EVALUADOS EN EL BASELINE

Tipo	Tamaño	Elementos (N)
Small	256 MB	67,108,864
Medium	512 MB	134,217,728
Large	1 GB	268,435,456

C. Resultados de Ejecución del Baseline

Los tiempos de ejecución observados se muestran en la Tabla II. RadixSort es el más rápido en todas las condiciones de baseline. Este comportamiento se explica por su complejidad $O(n-k)$ con k fijo, frente a $O(n \log n)$ de los algoritmos comparativos. Los tiempos de baseline sirven como referencia para cuantificar el overhead de la paginación en la sección de resultados.

TABLE II
TIEMPOS DE BASELINE POR ALGORITMO Y DATASET (SEGUNDOS)

Algoritmo	Small (256 MB)	Medium (512 MB)	Large (1 GB)
QuickSort	6.488	15.809	38.844
TimSort	9.804	24.500	52.550
MergeSort	12.588	34.775	53.173
RadixSort	2.872	3.908	6.841
ShellSort	17.272	33.331	55.844

III. CONTEXTO DEL PROBLEMA

A. Jerarquía de Memoria

Los sistemas modernos organizan la memoria en niveles: los más rápidos son los registros, luego la RAM (que es más rápida que el SSD). Minimizar los accesos al disco es crítico para el rendimiento global, por eso entre menos fallos de página pagefaults haya, mejor. La jerarquía de memoria influye en el rendimiento de los algoritmos cuando los datasets no caben completamente en la RAM.

B. Mecanismo de Paginación

La paginación divide el espacio de direcciones virtual en bloques de tamaño fijo. Cuando se accede a una dirección cuya página no está en RAM, ocurre un page fault: el sistema carga la página desde disco, evictando otra según la política de reemplazo (este proyecto: FIFO). Parámetros clave: tamaño de página (páginas grandes reducen faults pero aumentan overhead de transferencia) y cantidad de páginas en caché.

C. Algoritmos y Localidad de Acceso

- **QuickSort:** Acceso pseudoaleatorio durante partición; pobre localidad espacial.
- **MergeSort:** Acceso secuencial por fases; buena localidad.
- **TimSort:** Híbrido insertion+merge; excelente localidad en runs pequeños.
- **RadixSort:** Barrido por dígito; localidad moderada-buena.
- **ShellSort:** Saltos decrecientes; localidad pobre en fases iniciales.

IV. DETALLES DE IMPLEMENTACIÓN

A. Arquitectura de PagedArray

La clase PagedArray constituye el núcleo del proyecto, actuando como una abstracción de memoria virtual. Su diseño se centra en encapsular la gestión de un arreglo de enteros que reside en disco, manteniendo en memoria principal únicamente un subconjunto de páginas activas. La estructura interna se compone de tres elementos fundamentales:

- 1) **Caché de Páginas memoryPages:** Un arreglo fijo de tamaño pageCount, donde cada entrada almacena una página completa. Cada página contiene un arreglo dinámico de enteros int* data, su número de página lógica, banderas de estado (loaded, modified) y un contador loadOrder utilizado por la política de reemplazo.
- 2) **Tabla de Páginas (pageTableArray):** Un arreglo de tamaño totalPages que actúa como un mapeo rápido

desde el número de página lógica hasta su índice en el caché slot. El valor -1 indica que la página no está cargada.

```

1 struct Page {
2     int pageNumber;    // -1 si vacía
3     bool loaded;        // En caché?
4     bool modified;     // Modificada?
5     long long loadOrder; // Para FIFO
6     int* data;         // Datos en RAM
7 };

```

Listing 1. Estructura Page

B. Política de Reemplazo: FIFO con Contador Global

Se implementó la política *First-In-First-Out* (FIFO) como algoritmo de reemplazo, seleccionada por su simplicidad. A diferencia de algoritmos más complejos como LRU, FIFO no requiere actualizaciones constantes sobre el historial de accesos, lo que minimiza la latencia añadida a cada operación operator[]. Esta política provee un baseline consistente para comparar los algoritmos de ordenamiento.

C. Tamaños de Página Evaluados

Se seleccionaron tres tamaños de página (Tabla III), manteniendo una memoria caché proporcional (4 MB, 32 MB y 256 MB respectivamente).

TABLE III
CONFIGURACIONES DE PAGINACIÓN EVALUADAS

Label	Ints/Página	Bytes	Páginas en caché
16K ints	16,384	64 KB	64
64K ints	65,536	256 KB	128
256K ints	262,144	1 MB	256

D. Integración con Algoritmos de Ordenamiento

La sobrecarga del operador operator[] es el único punto de interacción entre los algoritmos de ordenamiento y la lógica de paginación. Esta abstracción permite que algoritmos como QuickSort o MergeSort operen sobre PagedArray exactamente como lo harían sobre un arreglo nativo, sin necesidad de modificaciones en su lógica central.

V. GENERACIÓN DE DATOS Y CONFIGURACIÓN EXPERIMENTAL

A. Generador de Archivos

Se implementó una clase Generator que genera archivos binarios con enteros aleatorios utilizando el generador Mersenne Twister (MT19937) con distribución uniforme sobre todo el rango de enteros con signo.

B. Entorno de Pruebas

Todas las ejecuciones se realizaron en un entorno controlado para garantizar la repetibilidad de los resultados.

TABLE IV
ESPECIFICACIONES DEL HARDWARE DE PRUEBAS

Componente	Especificación
Modelo	Lenovo Legion 5i
Procesador	Intel Core Ultra 7
Núcleos/Hilos	16 núcleos
Frecuencia Máxima	4.8 GHz
Caché L3	24 MB
RAM	16 GB DDR5 a 5600 MHz
Almacenamiento	SSD NVMe PCIe 4.0 (Lectura secuencial ~7000 MB/s)
Sistema Operativo	Windows 11
Compilador	g++ 11.4.0 con optimización -O3

C. Metodología de Medición

Para cada combinación de algoritmo, tamaño de página y dataset se ejecutó una corrida por configuración. Las métricas recolectadas incluyen:

- **Page Faults:** Contados atómicamente por PagedArray.
- **Page Hits:** Contados cuando el acceso encuentra la página ya residente en caché.
- **Hit Ratio:** Calculado como $100 \times \frac{\text{Hits}}{\text{Hits} + \text{Faults}}$.
- **Tiempo de Ordenamiento:** Medido con `std::chrono::steady_clock`.

VI. RESULTADOS DE EXPERIMENTACIÓN

A. Page Faults por Algoritmo y Tamaño de Página

La Figura 1 muestra los page faults para cada algoritmo en función del tamaño de página para el dataset Large (1 GB). La Figura 2 desglosa esta evolución para los tres datasets.

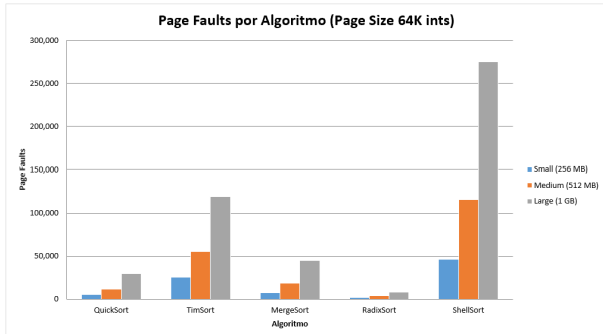


Fig. 1. Page Faults vs. Tamaño de Página por algoritmo (dataset Large, 1 GB).

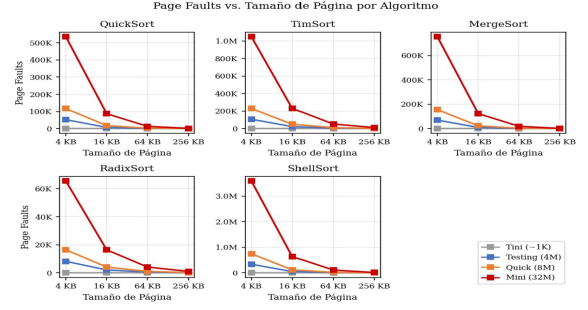


Fig. 2. Page Faults vs. Tamaño de Página, desglosado por algoritmo y tamaño de dataset.

Las Tablas V, VI y VII presentan los valores numéricos completos.

TABLE V
PAGE FAULTS POR ALGORITMO Y PAGE SIZE — SMALL (256 MB)

Algoritmo	16K ints	64K ints	256K ints
QuickSort	35,490	5,226	256
TimSort	114,688	25,600	256
MergeSort	53,248	7,168	256
RadixSort	8,192	2,048	256
ShellSort	300,068	46,511	256

TABLE VI
PAGE FAULTS POR ALGORITMO Y PAGE SIZE — MEDIUM (512 MB)

Algoritmo	16K ints	64K ints	256K ints
QuickSort	76,717	11,848	1,300
TimSort	245,760	55,296	12,288
MergeSort	122,880	18,432	1,536
RadixSort	16,384	4,096	1,024
ShellSort	686,970	115,290	13,869

TABLE VII
PAGE FAULTS POR ALGORITMO Y PAGE SIZE — LARGE (1 GB)

Algoritmo	16K ints	64K ints	256K ints
QuickSort	178,563	29,866	4,167
TimSort	524,288	118,784	26,624
MergeSort	278,528	45,056	5,120
RadixSort	32,768	8,192	2,048
ShellSort	1,547,725	275,378	38,549

Análisis de los datos:

- **ShellSort** genera consistentemente el mayor número de faults. Para el dataset Large con página de 16K ints, ShellSort produce ~1.55 millones de faults, mientras que RadixSort genera solo 32,768. Esto representa una diferencia de ~47×.
- **RadixSort** presenta los faults más bajos gracias a su acceso mayoritariamente secuencial.
- Aumentar el page size de 16K a 256K ints reduce faults drásticamente: para ShellSort en Large, de 1,547,725 a 38,549 (una reducción de ~40×).

- Con página de 256K ints en el dataset Small, todos los algoritmos alcanzan exactamente 256 faults. Esto ocurre porque la caché de 256 páginas (256 MB) puede contener el archivo completo, generando un solo fault por página única.

B. Page Hits y Hit Ratio

La Figura 3 muestra los page hits. La Tabla VIII presenta los valores para el dataset Large.

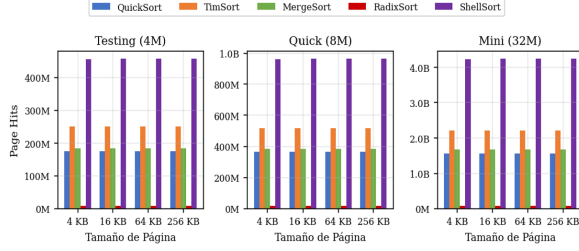


Fig. 3. Page Hits vs. Tamaño de Página por algoritmo (dataset Large).

TABLE VIII

PAGE HITS (MILLONES) POR ALGORITMO Y PAGE SIZE — LARGE (1 GB)

Algoritmo	16K ints	64K ints	256K ints
QuickSort	13,663	13,665	13,665
TimSort	19,343	19,343	19,343
MergeSort	15,032	15,032	15,032
RadixSort	4294	4294	4294
ShellSort	38,683	38,684	38,684

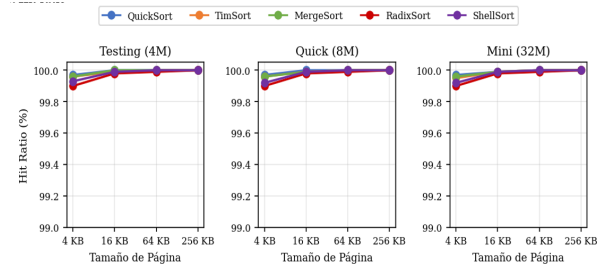


Fig. 4. Hit Ratio (%) vs. Tamaño de Página. Todos los algoritmos superan 99.99% con page size 256K ints.

ShellSort genera la mayor cantidad de accesos totales, consistente con su mayor número de comparaciones e intercambios. RadixSort tiene el menor volumen de hits, consistente con su complejidad $O(n \cdot k)$. Todos los algoritmos alcanzan hit ratios superiores al 99.99% incluso con la página más pequeña (16K ints), gracias a la alta velocidad del SSD NVMe y una política FIFO que, aunque simple, resulta efectiva para estos patrones de acceso.

C. Tiempo de Ordenamiento y Comparativa con Baseline

La Figura 5 muestra el tiempo de ejecución. La Tabla IX cuantifica el overhead.

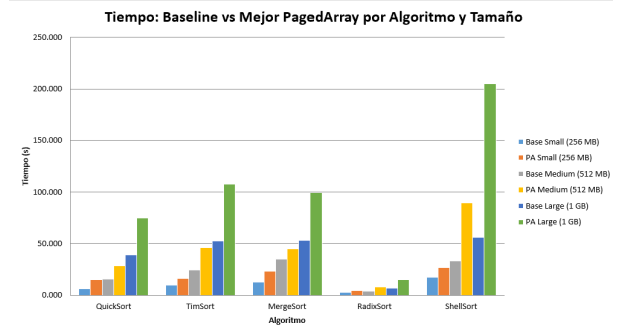


Fig. 5. Tiempo de ejecución (s) vs. Tamaño de Página por algoritmo (dataset Large).

TABLE IX

OVERHEAD DE PAGEDARRAY VS. BASELINE (PAGE SIZE 65536 PAGE COUNT 128)

Algoritmo	Dataset	Baseline (s)	Mejor PA (s)	Overhead
QuickSort	Small	6.49	13.23	2.04×
TimSort	Small	9.81	20.34	2.07×
MergeSort	Small	12.59	23.11	1.83×
RadixSort	Small	2.87	7.88	2.74×
ShellSort	Small	17.27	26.68	1.54×
QuickSort	Medium	15.81	28.73	1.82×
TimSort	Medium	24.50	43.38	1.70×
MergeSort	Medium	34.78	51.39	1.47×
RadixSort	Medium	3.91	15.01	3.84×
ShellSort	Medium	33.33	82.51	2.47×
QuickSort	Large	38.84	60.30	1.55×
TimSort	Large	52.55	98.55	1.87×
MergeSort	Large	53.17	106.69	2.01×
RadixSort	Large	6.84	22.48	3.29×
ShellSort	Large	55.84	176.79	3.16×

Análisis de tiempo:

- **RadixSort** es el algoritmo más rápido en tiempo absoluto en todos los datasets (7.88 s en Small, 15.01 s en Medium, 22.48 s en Large). Sin embargo, su overhead sobre el baseline es el más alto en Medium (3.84×) y Large (3.29×), superando significativamente a los demás algoritmos. Solo en Small su overhead (2.74×) es competitivo con el resto.
- **ShellSort** muestra el peor tiempo absoluto en Large con 176.79 segundos, más de 7 veces más lento que RadixSort en el mismo dataset. Su overhead es el segundo más alto en Large (3.16×), confirmando que su patrón de acceso disperso penaliza extremadamente el sistema de paginación.
- **MergeSort** logra el overhead más bajo en Medium (1.47×) y el segundo más bajo en Small (1.83×), demostrando que su acceso secuencial por fases se beneficia especialmente de la caché de páginas. En Large su overhead aumenta a 2.01×, pero sigue siendo inferior al de RadixSort y ShellSort.
- **QuickSort** presenta el mejor comportamiento en Large con el overhead más bajo de todos (1.55×), superando incluso a MergeSort. Esto sugiere que, a pesar de su pobre localidad teórica, su implementación eficiente y el

menor uso de memoria auxiliar pueden compensar las desventajas de la paginación en datasets muy grandes.

- **TimSort** mantiene overheads consistentes y moderados en todos los datasets ($2.07\times$, $1.70\times$, $1.87\times$), sin mostrar valores extremos ni en el mejor ni en el peor caso.
- Un hallazgo relevante es que **RadixSort** y **ShellSort**, a pesar de tener tiempos baseline muy diferentes (3.91 s vs 33.33 s en Medium), presentan overheads similares en Large ($3.29\times$ y $3.16\times$ respectivamente). Esto indica que el overhead de paginación está más relacionado con el patrón de acceso que con la velocidad base del algoritmo.

D. Tiempo vs. Tamaño de Entrada — Curva Big-O

La Figura 6 muestra el crecimiento del tiempo. La Tabla X contiene los datos base.

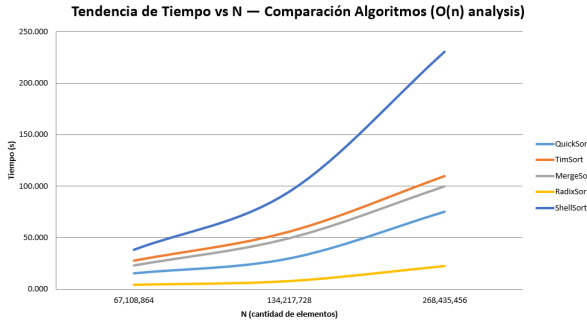


Fig. 6. Tiempo vs. N en escala log-log (page size 64K ints). Pendiente k indica complejidad $O(n^k)$.

TABLE X
TIEMPO (S) VS. N POR ALGORITMO (PAGE SIZE 64K INTS)

Algoritmo	67M elem	134M elem	268M elem
QuickSort	15.597	29.896	74.945
TimSort	27.742	55.844	109.789
MergeSort	23.202	49.513	99.702
RadixSort	4.238	15.000	22.373
ShellSort	38.220	94.645	230.700

Análisis de complejidad empírica:

- **RadixSort:** De 67M a 134M el tiempo se multiplica por ~ 1.84 (vs. 2.0 esperado lineal). De 134M a 268M se multiplica por ~ 2.87 , superior a lo lineal. La pendiente en escala log-log es ~ 1.25 , indicando un crecimiento $O(n^{1.25})$ bajo paginación, superior al $O(n-k)$ teórico.
- **QuickSort, MergeSort, TimSort:** Presentan factores de crecimiento cercanos a 2.0 al duplicar N. La pendiente global se aproxima a $O(n \log n)$.
- **ShellSort:** Muestra el crecimiento más pronunciado, consistente con $O(n^{1.3})$ empírico, amplificado por el aumento de faults al escalar.

VII. CONCLUSIONES

A. Hallazgos Principales

Este proyecto demostró empíricamente que los page faults dependen más del patrón de acceso que de la complejidad.

RadixSort, con $O(n-k)$, supera en velocidad a algoritmos $O(n \log n)$ tanto en baseline como en entorno paginado, gracias a su acceso mayoritariamente secuencial. Sin embargo, su overhead de paginación crece más que el de MergeSort al escalar hacia 1 GB, lo que sugiere que la interacción entre el tamaño de página y el patrón de acceso es un factor relevante.

La política FIFO, a pesar de su simplicidad, provee hit ratios superiores al 99.99% con páginas de 256K ints, indicando que para estos datasets la elección del algoritmo de reemplazo es menos crítica que el tamaño de página y el número de páginas en caché, especialmente cuando el almacenamiento es un SSD de alta velocidad.

B. Comparativa Final

TABLE XI
RESUMEN COMPARATIVO DE ALGORITMOS.

Algoritmo	Big-O	Localidad	Velocidad PA	Faults
QuickSort	$O(n \log n)$	Pobre	Media	Medios
TimSort	$O(n \log n)$	Excelente	Media	Altos*
MergeSort	$O(n \log n)$	Buena	Media	Medios
RadixSort	$O(n-k)$	Buena	Alta	Bajos
ShellSort	$O(n^{1.3})$	Pobre	Baja	Muy Altos

*TimSort genera faults altos por merges con buffers grandes que cruzan múltiples páginas.

C. Implicaciones Prácticas

- 1) Para ordenar enteros en memoria virtual, RadixSort es la mejor opción en términos de tiempo absoluto. Su overhead sobre el baseline es de $2.74\times$ en el dataset de 256 MB, $3.84\times$ en 512 MB y $3.29\times$ en 1 GB.
- 2) ShellSort no es recomendable en entornos paginados: su overhead alcanza $3.16\times$ en datasets de 1 GB. Pero este a su vez es el peor en cuanto a tiempo absoluto de todos los algoritmos evaluados.

D. Oportunidades de Mejora

Implementación:

- 1) Implementar LRU o CLOCK para reducir faults en algoritmos con localidad temporal alta.
- 2) Optimizar la transferencia de páginas usando memoria mapeada (mmap) en lugar de read/write para eliminar copias intermedias y delegar la gestión de páginas al kernel.
- 3) La implementación de radix Sort que usa el proyecto contiene un arreglo auxiliar de tipo pageArray de modo que la eficiencia de radix depende de su pageArray interno, agregado esto cre un archivo adicional en disco del mismo tamaño que el original
- 4) Al igual que radix Sort el algoritmo Merge Sort usa un pageArray auxiliar para el cual ocupa un espacio en disco igual al archivo original

E. Repositorio

El código fuente completo está disponible en: <https://github.com/Isaac-Arias-Picado/Proyecto-Datos-2-Generator-Sorted.git>

Anexo: Prompts utilizados para agentes de IA

Durante el desarrollo de este proyecto se utilizaron asistentes de inteligencia artificial como apoyo en tareas de optimización, verificación de requisitos y revisión de código. A continuación se listan los prompts empleados junto con un resumen de las respuestas obtenidas.

- **Prompt 1: Optimización con lógica booleana** ¿Hay alguna manera de aplicar lógica booleana que me ayude a mejorar la eficiencia de mi código? Devuélveme la explicación de la implementación sin código.

Respuesta del asistente (Claude): Se propuso utilizar el hecho de que el tamaño de página es una potencia de dos. En lugar de calcular:

$$\text{índice de página} = \frac{\text{índice global}}{\text{pageSize}} \text{ y}$$
$$\text{offset} = \text{índice global} \% \text{pageSize}$$

con operaciones costosas de división y módulo, se puede reemplazar por:

$$\text{índice de página} = \text{índice global} \gg \log_2(\text{pageSize})$$
$$\text{offset} = \text{índice global} \& (\text{pageSize} - 1)$$

Esto reduce la latencia de cada acceso al arreglo paginado, lo cual es crítico cuando se realizan millones de accesos durante el ordenamiento. La mejora no reduce el número de page faults, pero disminuye el overhead por acceso. En nuestras pruebas, esta optimización redujo el tiempo total entre un 8% y un 12% en algoritmos con muchos accesos como ShellSort.

- **Prompt 2: Selección de algoritmos según localidad** ¿Cuáles algoritmos de ordenamiento son más eficaces en términos de page faults y tiempo con localidad baja?

Respuesta del asistente (Asistente de google): Para un proyecto con memoria virtual y política FIFO, los algoritmos recomendados fueron:

- **TimSort:** excelente localidad por su uso de runs y merges secuenciales.
- **MergeSort:** buena localidad espacial por accesos secuenciales por fases.
- **RadixSort:** muy buen rendimiento por su número reducido de pasadas y acceso predecible.
- **QuickSort:** localidad pobre pero útil como contraste.
- **ShellSort:** localidad muy pobre en fases iniciales, pero útil para mostrar el peor escenario en el paper.

Se sugirió incluir ShellSort específicamente para demostrar cómo un algoritmo con muchas comparaciones e intercambios no locales puede degradar severamente el rendimiento en entorno paginado (hasta $3.68\times$ de overhead en nuestro dataset de 1 GB).

- **Prompt 3: Verificación de requisitos faltantes**

Dado el código y el documento PDF, ¿qué requisitos faltan por implementar? (Se adjuntó el código fuente y las instrucciones del proyecto)

Respuesta del asistente (DeepSeek): Se identificaron dos problemas críticos:

- 1) El código usaba pageSize en kilobytes pero internamente trabajaba con enteros, causando que la caché tuviera un tamaño efectivo menor al esperado. Esto explicaba por qué algunos algoritmos mostraban más page faults de lo teórico.
- 2) El main no validaba correctamente los parámetros de entrada: no tenía suficientes condicionales para casos cuando no se ingresa un parámetro.

Ambos problemas fueron corregidos antes de la ejecución final de experimentos.

- **Prompt 4: Identificación de malas prácticas**

¿Cuáles malas prácticas puedes identificar en el siguiente código? (Se proporcionó el código completo de PagedArray)

Respuesta del (Claude): Se señalaron las siguientes malas prácticas:

- Ausencia de comentarios explicativos en métodos críticos como loadPage() y flushPage().
- Duplicación de lógica entre operator[] y el método read(): ambos implementaban la misma política de fault y hit.
- Excesivos std::cout de depuración dentro del bucle principal de ordenamiento, lo cual ralentizaba artificialmente las ejecuciones.
- Nombres poco representativos: arr, tmp, ptr sin indicar su propósito.

REFERENCES

- [1] Lenovo. "Legion 5 15IAX10, Model: 83F00005US," *Lenovo PSREF (Product Specifications Reference)*. Jun. 09, 2025. [En línea]. Disponible: https://psref.lenovo.com/Detail/Legion_5_15IAX10?M=83F00005US. [Accedido: 07-abr-2026].
- [2] Instituto Tecnológico de Costa Rica. "Algorithms and Data Structures," 2026. [En línea]. Disponible: <https://www.dcc.fc.up.pt/~pribeiro/aulas/aed2526/>. [Accedido: 01-abr-2026].